

SmolVLM: Redefining Small and Efficient Multimodal Models

Hugging Face & Stanford University

A family of compact vision-language models achieving competitive performance with dramatically lower memory footprints through strategic architectural optimizations.

SmolVLM-256M • <1 GB RAM • Outperforms 300× larger Llafics-80B

Large VLMs Are Powerful but Impractical for Edge

Deployment

State-of-the-art VLMs demand massive compute and memory, restricting deployment to cloud GPUs.

Molmo-A1B-7B requires 27.7 GB GPU RAM — far beyond smartphone or edge-device limits.

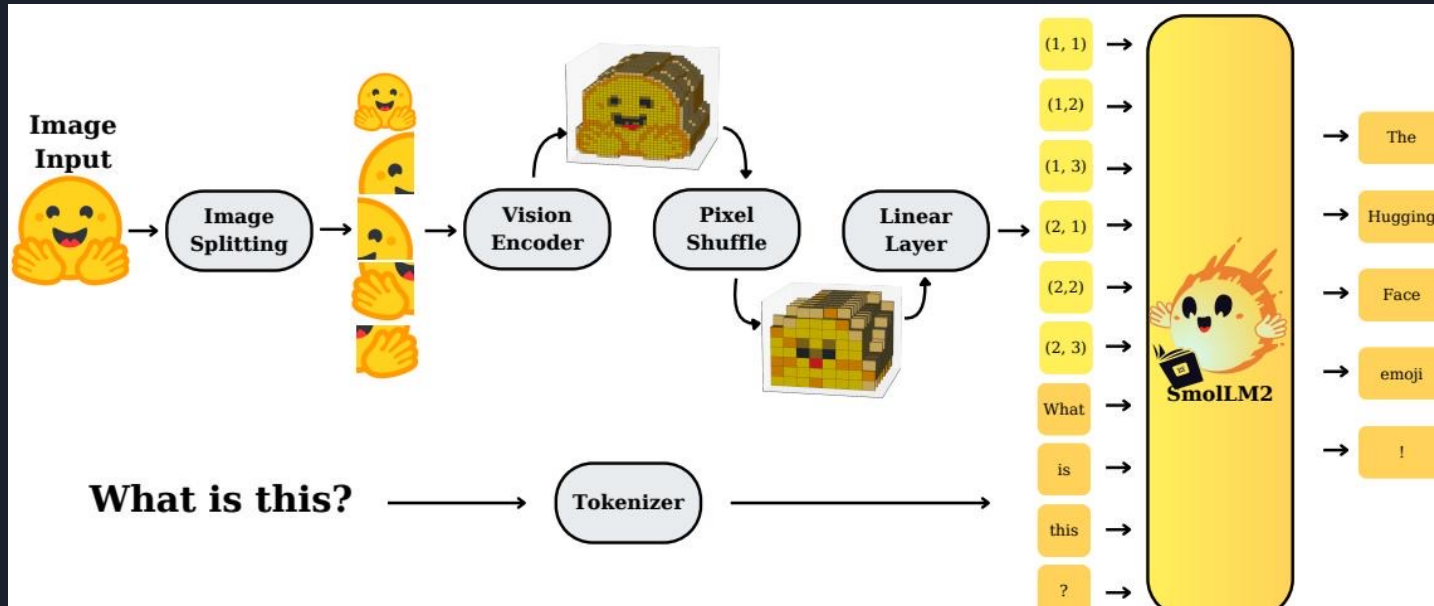
Existing 'efficient' open-source models still consume multiple gigabytes, leaving a gap for true on-device inference.

Mobile and IoT applications need sub-1GB multimodal models without sacrificing core capabilities.

Memory Footprint Comparison

Molmo-A1B-7B	27.7 GB
InternVL2-2B	~6–8 GB
SmolVLM-2.2B	4.9 GB
SmolVLM-500M	1.2 GB
SmolVLM-256M	0.8 GB

SmolVLM Architecture: Image Splitting, Pixel Shuffle, and SmoLM2 Backbone



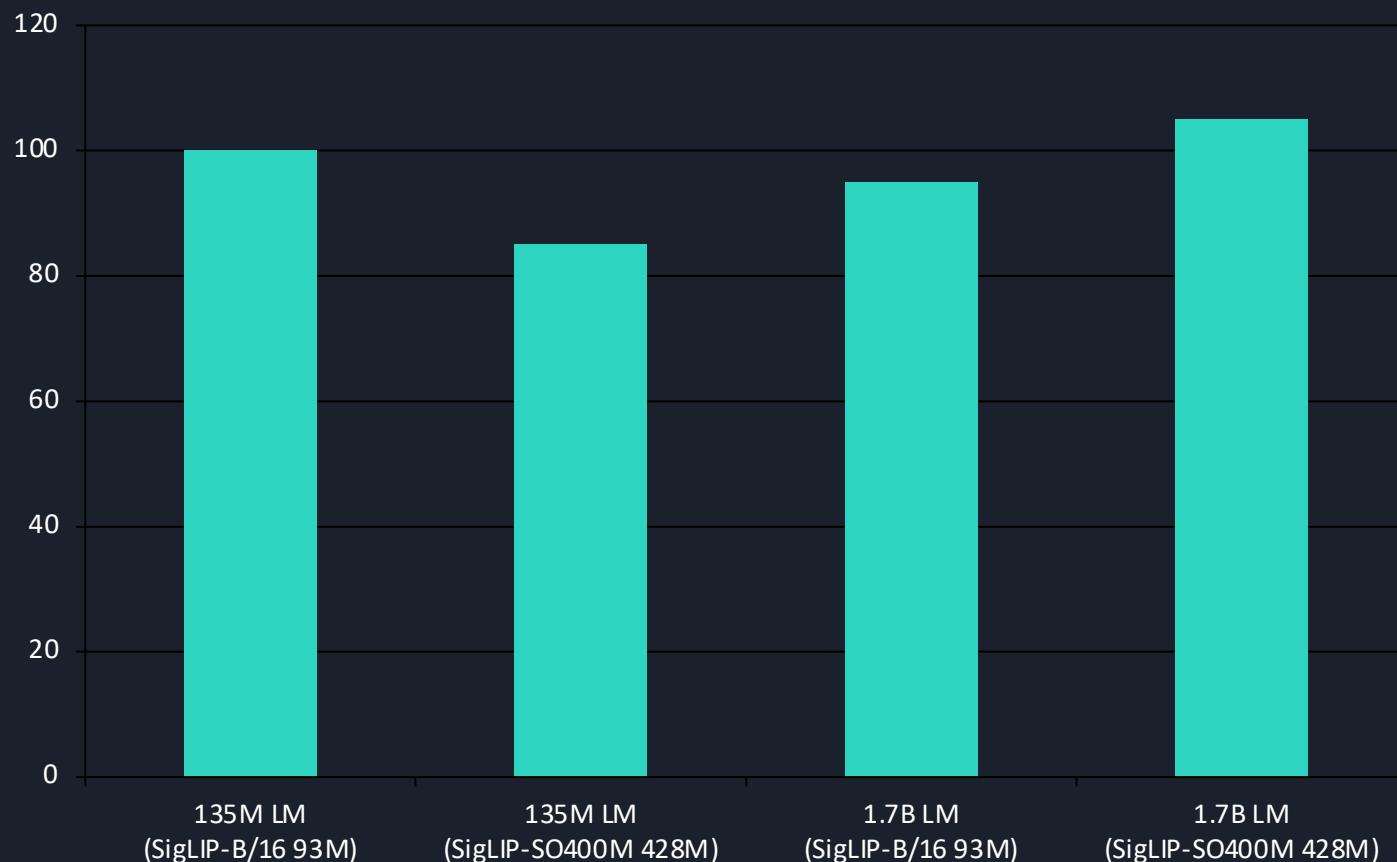
1. Image Splitting – Input image is tiled into sub-images for local processing.
2. Vision Encoding – Each tile is encoded by a compact SigLIP vision encoder.
3. Pixel Shuffle – Spatial features are rearranged into channels, compressing tokens by r^2 .
4. Linear Projection – Compressed visual features are projected into SmoLM2's input space.
5. Text Concatenation – Visual and text token embeddings are fused and fed to the LM.

Smaller Vision Encoders Outperform Larger Ones in Compact

VLMs

Finding 1 — Balanced Encoder-LM Compute: a 93M SigLIP-B/16 paired with a 135M LM outperforms a 428M SigLIP-SO400M.

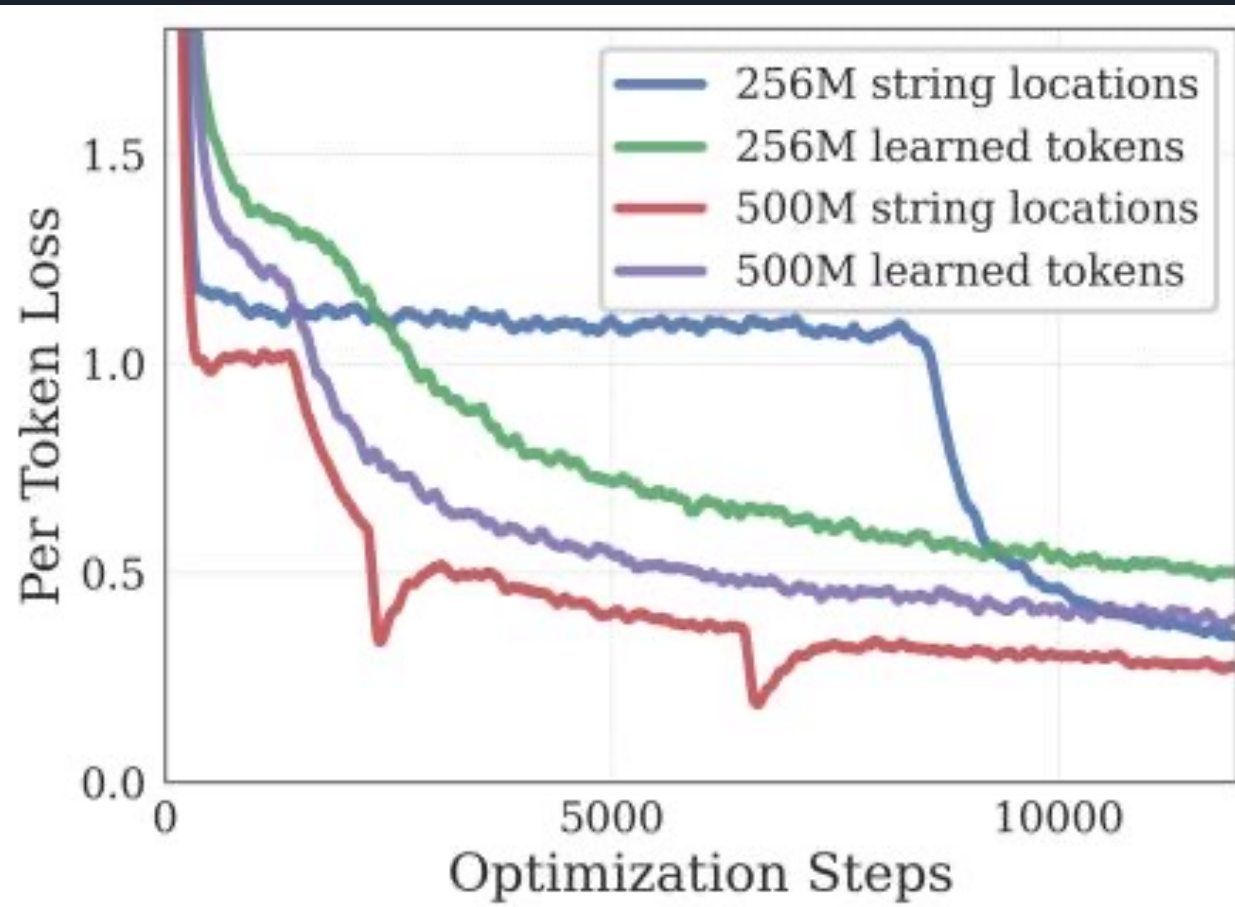
Encoder-LM Performance Trade-off



Key Insight

- For small LMs ($\leq 135\text{M}$), the 93M SigLIP-B/16 yields better end-to-end performance than the 428M encoder.
- The larger SigLIP-SO400M only justifies its $\sim 10\%$ parameter overhead at 1.7B LM scale.
- Compact VLMs benefit from balanced compute: avoid over-investing in vision encoders when the LM is small.

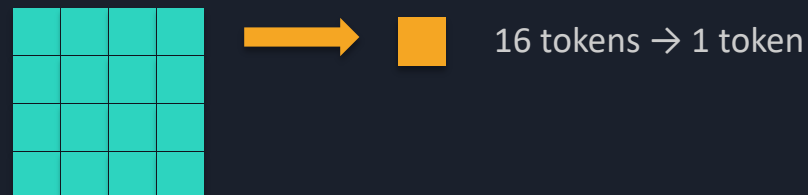
Aggressive Pixel Shuffle (r=4) Cuts Visual Tokens 16× for Small Models



Pixel Shuffle Mechanism

- Spatial features are rearranged into depth (channel) dimension.
- Reduction factor = r^2 : $r=2 \rightarrow 4\times$ fewer tokens; $r=4 \rightarrow 16\times$ fewer tokens.
- Small VLMs (256M/500M) benefit most from $r=4$, easing attention overhead.
- Larger models (2.2B) prefer $r=2$, preserving finer spatial detail.

Token Reduction at $r=4$



Extending Context from 2K to 16K Tokens Unlocks Higher Image Resolution

Finding 2 — Extended Context Length: performance improves significantly when scaling from 2K to 16K tokens.

Mechanism

- RoPE base increased from 10,000 → 273,000 to support longer sequences.
- Fine-tuned on mixed long/short-context data for stable extrapolation.
- 2.2B model uses 16K context; 256M/500M variants use 8K.
- Enables processing of higher-resolution images and longer video clips without truncation.

Impact

- Higher image resolution directly improves OCR and chart QA accuracy.
- Longer video context boosts Video-MME and MLVU scores.
- Attention overhead remains manageable thanks to pixel-shuffle token compression.
- Context scaling is complementary to aggressive compression — both are essential.

Context Scale: 2K → 8K → 16K

|

RoPE Base: 10K → 273K

SmolVLM Achieves Competitive Performance at Sub-1GB to 4.9GB

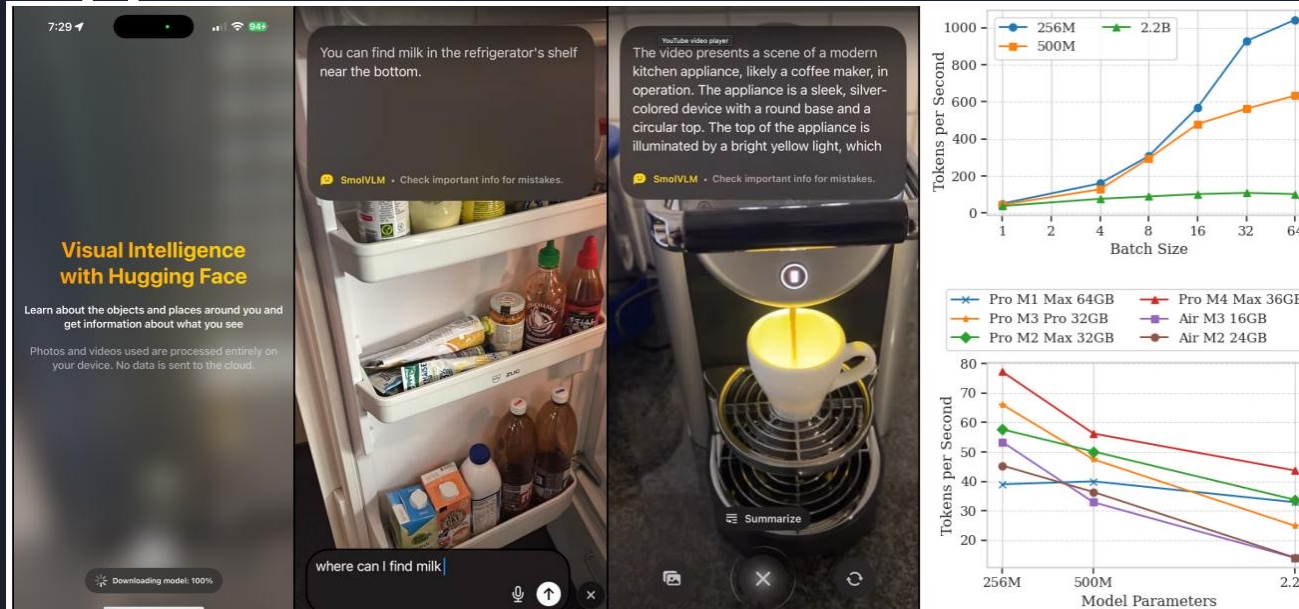
RAM

Evaluation across 13 benchmarks (Single-Image, Multi-task, Video) vs. Best Efficient OS baselines

Capability	Benchmark	SmolVLM 256M	SmolVLM 500M	SmolVLM 2.2B	Best Efficient OS
Single-Image	OCRBench	52.6%	61.0%	72.9%	54.7% (Molmo-A1B-7B)
Single-Image	A12D	46.4%	59.2%	70.0%	71.0% (Molmo-A1B-7B)
Single-Image	ChartQA	55.6%	62.8%	68.7%	48.0% (Molmo-A1B-7B)
Single-Image	TextVQA	50.2%	60.2%	73.0%	61.5% (Molmo-A1B-7B)
Single-Image	DocVQA	58.3%	70.5%	80.0%	77.7% (Molmo-A1B-7B)
Single-Image	ScienceQA	73.8%	80.0%	89.6%	87.5% (Molmo-A1B-7B)
Multi-task	MMMU	29.0%	33.7%	42.0%	33.9% (Molmo-A1B-7B)
Multi-task	MathVista	35.9%	40.1%	51.5%	37.6% (Molmo-A1B-7B)
Multi-task	MMStar	34.6%	38.3%	46.0%	43.1% (Molmo-A1B-7B)
Video	Video-MME	33.7%	42.2%	52.1%	45.0% (InternVL2-2B)
Video	MLVU	40.6%	47.3%	55.2%	48.2% (InternVL2-2B)
Average	All Benchmarks	44.0%	51.0%	59.8%	—

RAM Usage: SmolVLM-256M = 0.8 GB | SmolVLM-500M = 1.2 GB | SmolVLM-2.2B = 4.9 GB | Molmo-A1B-7B = 27.7 GB

Real-Time Inference on iPhones: Up to 80 Tokens/Second on Apple Silicon



Throughput Benchmarks

~80 tokens/sec

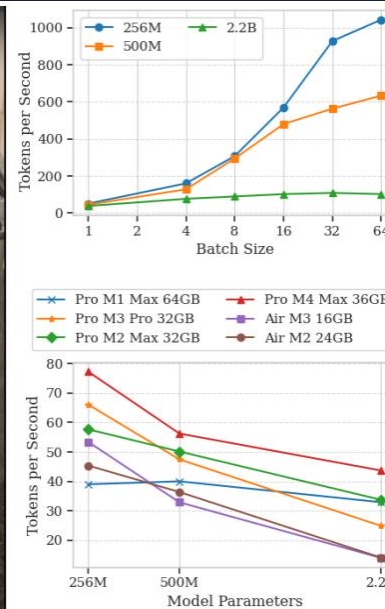
Pro M1 Max 64GB
(batch=1)

~1,000 tokens/sec

Pro M1 Max 64GB
(batch=64)

~55 tokens/sec

MacBook Air M3 16GB
(batch=1)



HuggingSnap iOS App

Photos and videos processed entirely on-device — zero cloud dependency.

Key Takeaways: Design Principles for Efficient Multimodal Models

1

Balance Encoder-LM Compute

For small LMs, a 93M SigLIP-B/16 outperforms a 428M encoder. Scale encoder size with LM capacity.

2

Extend Context Length

Increase RoPE base (10K → 273K) and fine-tune on mixed data. 16K context unlocks higher-resolution inputs.

3

Aggressive Token Compression

Small VLMs benefit from $r=4$ pixel shuffle (16× reduction). Larger models prefer $r=2$ for finer detail.

4

Sub-1GB Inference Is Achievable

SmolVLM-256M runs in 0.8 GB RAM and surpasses the 300× larger LLaVA-80B.

5

Open-Source Everything

All weights, datasets, code, and the HuggingSnap mobile app are fully open-sourced for reproducibility.

Implication: Architectural efficiency matters more than raw scale for on-device multimodal AI.